AI 辅助编程范式与工程实践 — 学习笔记

> 目的：厘清常见 **coding 方式** 的区别、使用规范、适用场景与应用层写法（含 Prompt 分层），并覆盖**面试高频与进阶问答。**
> 范围侧重 **AI 时代** 的提法（Vibe / Spec / Context / Harness / Agentic）及与传统 **TDD** 的对照；不构成对任何厂商营销用语的背书。

一、为什么要区分「方式」

- **同一种工具**（如 Cursor），不同用法会带来完全不同的结果：**速度、可维护性、合规与事实准确性。**
- **面试进阶**常考： trade-off（速度 vs 可维护）、**失败模式**（何时胡编、何时越权）、**在 SDLC 中规格放哪**、**如何验收 AI 产出。**

下面用一张心智图串起概念：

```
Harness（缰绳）      → 能做什么、不能做什么、失败怎么办
        ↓
Spec（规格）         → 什么叫「完成」、验收是什么
        ↓
Context（上下文）    → 本轮必须看见哪些材料
        ↓
Agentic（多步）      → 谁先谁后、工具链与阶段
        ↓
Vibe（表达）         → 在事实与规格已满足的前提下，如何写得好读、好维护
```

注意：这不是严格的「进化链」——**不是** Harness 一定比 Context「更先进」，而是**不同维度**；复杂项目里往往**五者同时需要。**

二、概念速查表（面试友好）

名称	一句话	解决什么问题	典型载体
Vibe Coding	自然语言描述目标，快速迭代，少前期结构	探索、原型、个人脚本、学习	对话式改代码
Spec-Driven / Spec Coding	**先**把需求、行为、验收写清楚，再实现	可维护、可协作、可对齐验收	PRD、用户故事、验收清单、契约测试
Context Engineering	精心选择**进入模型窗口**的信息（文件、摘要、规则）	减少胡编、提高命中率	@ 文件、Skills、检索片段

名称	一句话	解决什么问题	典型载体
Harness Engineering	为 Agent 设计**运行环境**：工具权限、规则、钩子、观测与回滚	生产级、多步自治、可控失败	CI、Rules、Skills、沙箱、审批点
Agentic Engineering	多步、多工具、可并行/串行的**代理流水线**	复杂任务自动化（仍常需人在环）	多 Agent、任务图、编排器
TDD（传统）	先写失败测试 → 最小实现 → 重构	设计可测、回归有保障	单元测试、红绿重构循环

辨析（易混点）

- **Prompt Engineering**：主要管**怎么说**（指令措辞）。
- **Context Engineering**：管**给什么材料**（输入侧）。
- **Harness Engineering**：管**整套运行时**（权限 + 流程 + 反馈），常被比喻为：模型像 CPU，上下文像 RAM，**Harness 像操作系统**（社区比喻，非严谨类比）。

三、贯穿案例：同一句话，不同「coding 方式」怎么写?

原始一句话（偏 Vibe）：
「帮我设计一个好看的独立站页面。」

这句话**只有审美与意图**，没有受众、目标、技术栈、验收——适合快速试，不适合直接上生产。下面展示**同一需求**在六种口径下的**改写示例**与**收益 / 代价**（面试可举例）。

3.1 Vibe Coding（氛围 / 凭感觉）

Prompt 示例

```
帮我设计一个好看的独立站页面，风格简约高级，你直接生成 HTML/CSS 就行。
```

好处	代价 / 风险
最快看到视觉，适合**灵感与方向试探**	「好看」主观；难复现；难协作；易写出**不可维护的大块 HTML**
零准备成本	无验收标准；换一版 prompt 可能**整页推翻**

适用：个人练手、静态 demo、内部汇报稿。**不适用**：要品牌一致、要无障碍、要 SEO 的团队站点。

3.2 Spec-Driven（规格先行）

Prompt 示例

```
【Spec】做一个单页落地页（Above the fold + 特性三列 + CTA + 頁腳）。
【验收】① Lighthouse Performance ≥ 90（本地 build 后测）② 键盘可聚焦到 CTA ③ 文案中英可选④ 移动
【非目标】不做后端、不做多语言路由、不做动效库以外的动画。
【交付物】/src/pages/index.astro（或约定栈）+ README 如何 build。
```

好处	代价
可对齐验收，减少「我觉得不好看」扯皮	前置要想清范围，慢半拍
便于 Code Review：**对照 Spec 驳回**	Spec 写错会系统性做错，需迭代规格

适用：要上线、要给别人维护的落地页。**面试点**：Spec 是**合同**，不是作文。

3.3 Context Engineering（上下文工程）

Prompt 示例

```
在以下约束下改版落地页（已 @ 附件）：
- @design-tokens.json：主色、圆角、字号阶梯
- @copy.md：最终文案（禁止改事实与数据）
- @competitor-screenshots/：仅参考信息层级，禁止抄袭文案
输出：仅修改 components/hero 与全局 layout，其它文件不动。
```

好处	代价
品牌与文案可控，减少模型「自由发挥」	准备 token、文案、参考本身要花精力
多轮对话可**复用同一 Context 包**	Context 过时（旧 token）会带偏

适用：已有设计体系、有法务/品牌红线文案。**面试点**：Context 解决的是**输入质量**，不是「更会聊天」。

3.4 Harness Engineering（缰绳 / 运行时）

Prompt 示例

```
【Harness】仅在仓库内 public/ 与 src/components/landing/* 下修改；禁止读取或打印 .env；禁止执行
【流程】改前开分支 feat/landing；改后贴出 diff 摘要；若引入新依赖须说明许可证。
【失败】若 build 失败，回滚到上一 commit，只保留最小 diff。
```

好处	代价
防越权、防泄密、防一次对话毁掉仓库	需要维护规则与分支习惯
与 CI 结合可**自动化守门**	初期配置成本

适用：开源协作、公司仓库、任何「AI 有写权限」的场景。**面试点**：Harness 讨论的是**治理**，不是提示词技巧。

3.5 Agentic Engineering（多步代理）

Prompt 示例

```
按顺序执行，每步结束向我汇报再进入下一步：
1）根据 Spec 生成信息架构草图（标题层级）
2）生成组件树与 props 约定
3）实现静态页面
4）跑 Lighthouse 与 a11y 扫描，列出不达标项
5）只针对不达标项做小步修复（禁止重写全页）
```

好处	代价
复杂任务可分解，每步可检查	编排失败会「原地打转」；需人在环
易接入工具（构建、扫描、截图对比）	对 Harness 依赖高（否则乱调用工具）

适用：页面复杂、要接构建/扫描/截图流水线。**面试点**：Agentic 是**编排问题**，不是模型变大就自动会。

3.6 TDD / 测试约束（偏传统，但可落在 UI）

Prompt 示例

```
先写 Playwright：给定访问 / ，可见 h1 文案为 copy.md 中标题；CTA 可点击且 href 为 mailto:xxx。
实现时以测试通过为完成定义；禁止为通过测试写死 sleep>2s。
```

好处	代价
关键路径可回归（链接、表单、核心文案）	视觉「好不好看」测不全，需配合设计走查
重构时有安全网	E2E 脆弱时要维护选择器

适用：有明确行为契约的页面（表单、关键 CTA）。**面试点**：TDD 在 UI 上常是**关键路径 E2E + 少量单元**，不是一切像素级测。

3.7 六合一对比（面试一句话）

方式	独立站这句话的核心变化	你换来了什么
Vibe	几乎不改写	速度；失去可控性
Spec	补验收与非目标	可对齐、可维护
Context	补设计 token 与文案	品牌与事实一致

方式	独立站这句话的核心变化	你换来了什么
Harness	补权限与 CI	安全与可回滚
Agentic	拆成多步 + 扫描	结构化与质量闭环
TDD	先测试关键行为	回归与重构勇气

进阶追问（面试官可能追问）：若只做 Vibe，**三个月后再改**会怎样？——典型答法：无 Spec、无测试、无 token，**只能靠重聊或重写**，技术债指数上升。

四、传统 TDD 是什么（面试常考）

TDD（Test-Driven Development）：开发顺序是 **测试先行。**

1. **红**：写一个**会失败**的测试（定义下一步行为）。
2. **绿**：写**刚好让测试通过**的实现。
3. **重构**：在测试仍绿的前提下整理代码。

与「先写代码再补测试」的区别：TDD 里测试是**设计工具**，不只是事后回归。

适用：业务规则清晰、回归价值高、适合自动化的模块。
不适用：强探索期 UI、一次性脚本、或测试成本远高于价值的场景。

五、Spec 就是 SOP 吗？

- **不完全是。**
- **SOP** 多指**操作流程文档**（谁、何时、按何顺序做事）。
- **Spec** 在软件里更宽：**可被对照的规格**——含需求、验收、接口契约、测试即规格等。
- **一份写得好的 SOP**（含触发条件、输入输出、验收）**通常就是很好的 Spec 载体**，但 Spec 还可以是 API Schema、Given-When-Then 等不一定叫 SOP 的形式。

六、五层框架：何时想哪一层（应用规范）

建议在单次任务中按此**思考顺序**（不是死板顺序，但缺上层容易翻车）：

顺序	层级	先问自己的问题
1	**Harness**	能改哪些目录？能否联网？数字/事实必须来自哪里？失败了怎么办？
2	**Spec**	本轮「完成」的定义是什么？非目标是什么？
3	**Context**	模型必须读到哪些文件/日期/接口说明？
4	**Agentic**	先跑哪个脚本、再读哪个表、再写哪个输出？
5	**Vibe**	仅事实与验收已满足后，如何写得更清晰？

新项目尚不清楚栈时：不要一上来满配 Spec；改用 **「探索合同」**——本轮只产出**方案对比表 + 推荐 spike + 待决项**，等技术路线收敛后再加强 Spec 与 Harness。

七、应用层：Prompt 怎么写（模板）

不必每轮写全五段，但建议**心里过一遍五层。**

7.1 通用骨架

```
【Harness｜缰绳】范围、密钥与数据纪律、禁止行为。
【Spec｜规格】目标、日期/版本、验收标准、非目标。
【Context｜上下文】必读文件、路径、环境约束。
【Agentic｜多步】执行顺序（对应 Skill Step 或流水线）。
【Vibe｜发挥】文风、结构偏好（不替代事实）。
```

7.2 跨会话衔接

- **以磁盘文件为准**：是否已有某输出文件 = 上一步是否完成。
- **交接便签**（3 行）：日期 / 已完成与待办 / 注意事项。
- **主流程写进 Skill 或 SOP 索引**，避免每个新对话口头重讲。

八、什么时候用哪种方式（决策指南）

情境	优先策略
黑客松、个人小工具、快速试错	**Vibe** 为主，事后若要保留则**收编为 Spec**
团队功能、要给别人维护、要上线	**Spec** + **Harness**（规则/CI/权限）
模型总答偏、漏读关键文件	**Context**（@ 文件、Skill、摘要策略）
多步工具调用、要可控、要审计	**Harness** + **Agentic**（分阶段、观测、回滚点）
核心逻辑长期演进、怕改坏	**TDD** 或至少关键路径自动化测试
只有想法没有栈	**探索型 Spec**（对比 + spike + 决策记录），**勿**强定终极栈

九、与 Cursor 工程物的关系（实践）

工程物	主要承载的层级
`.cursor/rules/*.mdc`	**Harness**（全局行为底线）+ 少量 **Context** 纪律

工程物	主要承载的层级
`.cursor/skills/*/SKILL.md`	**Spec**（流程与验收）+ **Context**（读档顺序）+ **Agentic**（Step）
项目内 `SOP_时间表.md` 或 README	**Spec 索引** + 人机协作入口
脚本与 CI	**Harness**（可重复、可观测）

十、面试进阶：不止定义，要讲清 trade-off 与失败模式

10.1 高频简答（基础）

Q1：什么是 Vibe Coding？有什么风险？
A：用自然语言快速让 AI 生成与迭代代码，强调速度与尝试；风险是**可维护性差、事实不可追溯、安全与边界模糊。**适合原型，上生产前应收编为 Spec 与测试/规则。

Q2：Spec-Driven 和写 PRD 有什么区别？
A：核心一致：先定义行为与验收。Spec-Driven 更强调**可执行、可验证**（对接测试、lint、API 契约），PRD 若足够细也可视为 Spec。

Q3：Context Engineering 和 Prompt Engineering 区别？
A：前者优化**输入内容**（文件、检索、摘要）；后者优化**指令措辞**。缺 Context 时，Prompt 再漂亮也会胡编。

Q4：Harness Engineering 是什么？
A：为 Agent 设计**运行环境**：工具权限、规则、钩子、日志、失败策略；让多步任务在可控边界内跑，而不只是一次对话。

Q5：Harness 比 Context「更高级」吗？
A：**不是。**不同维度；生产系统通常**同时需要**好上下文和好缰绳。没有好上下文，Harness 再好也可能往错方向执行。

Q6：TDD 在 AI 编程里还有用吗？
A：有用。AI 加速写实现，**测试**仍可约束行为与防回归；适合核心业务逻辑。探索期可不强求，收敛后补上。

Q7：如何向面试官描述「我怎样控制 AI 写出可维护代码」？
A：规格先行（Spec）、关键路径测试、代码评审与 CI、规则与 Skill 固化流程、敏感操作用 Harness 限制；上下文精准投喂，避免让模型猜路径与数据。

Q8：「终极一条 Prompt 做出永不改的项目」可能吗？
A：**不现实。**应追求**清晰验收、可回归测试、可迭代**；用 Spec + Harness 降低返工，而非承诺零 bug。

10.2 进阶追问（中高级）

Q：AI 写代码快了，Code Review 要审什么不一样？
A：更关注 **Spec 是否被偷偷改窄/改宽**、**错误处理与边界**、**安全（注入、密钥）**、**可测试性**；不只看风格。AI 易生成「看起来完整」但边界假的代码。

Q：如何度量 AI 辅助开发的质量？
A：结合 **缺陷率、返工率、CI 通过率、MTTR**、以及 **Spec 变更次数**；单看速度会误导。

Q：什么时候不应 TDD？
A：探索极强、需求每小时变、或验证成本极高时；可改用**短生命周期原型 + 收敛后补测试**。

Q：Spec 腐烂怎么办？
A：把 Spec 放版本控制、与验收测试或 checklist 绑定；变更走**小步 PR**，避免口头改需求。

Q：多 Agent 与单 Agent + 好 Harness 的区别？
A：多 Agent 解决**分工与并行**；若没有清晰接口与 Harness，会放大混乱。**先.harness 再扩 agent** 往往更稳。

十一、延伸阅读（自建）

- 项目内可参考：`business-idea-exploration` Skill（商业点子分层探索 → 实施规格 → 开发长 Prompt）。
- 股票 SOP 与分层对应关系，以仓库中的 `SOP_时间表.md` 与各 `SKILL.md` 为准。

十二、一页复习卡

| 想快 | → 控制 Vibe，事后收编 | | 想稳 | → Spec + Harness | | 想准 | → Context | | 想自动 | → Agentic + Harness | | 想长期 | → 测试/契约 + Spec |

一句话：**先缰绳与验收，再材料与步骤，最后才是文笔与风格。**

文档为学习整理，术语随社区演进可能变化；面试作答时结合个人项目举例会更得分。